

Learning uv: The 20% That Matters

1 What uv Is and Why

uv is an extremely fast Python package and project manager written in Rust. It replaces `pip`, `pip-tools`, `virtualenv`, `poetry`, and `pyenv` in most workflows. In practice you use it for two things: managing virtual environments and installing dependencies, both far faster than the old tools.

2 Installation

```
1 # macOS / Linux
2 curl -LsSf https://astral.sh/uv/install.sh | sh
3
4 # Windows (PowerShell)
5 powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
6
7 # Verify
8 uv --version
```

3 The Core Workflow (Projects)

This is the 80% use case. Start a project, add dependencies, run code.

```
1 # Create a new project (makes pyproject.toml, .venv, etc.)
2 uv init my-project
3 cd my-project
4
5 # Add a dependency (auto-creates/updates the venv + lockfile)
6 uv add requests
7 uv add "pandas>=2.0"
8 uv add torch numpy
9
10 # Remove a dependency
11 uv remove requests
12
13 # Run a script inside the project's environment
14 uv run python main.py
15 uv run pytest
16
17 # Sync the environment to match pyproject.toml / uv.lock exactly
18 uv sync
```

Key idea: You almost never activate the venv manually. `uv run` executes commands inside the project environment automatically.

4 Key Files

- `pyproject.toml` — declares your dependencies (human-edited or via `uv add`).
- `uv.lock` — exact pinned versions for reproducibility (auto-managed, commit it).
- `.venv/` — the virtual environment (do NOT commit, add to `.gitignore`).

5 Managing Python Versions

```
1 # Install a specific Python version
2 uv python install 3.12
3
4 # Pin a project to a Python version
5 uv python pin 3.12
6
7 # List available / installed versions
8 uv python list
```

6 Virtual Environments (Manual Control)

When you want explicit control instead of project mode:

```
1 # Create a venv
2 uv venv                      # uses .venv by default
3 uv venv --python 3.12
4
5 # Activate (traditional way)
6 source .venv/bin/activate    # macOS/Linux
7 .venv\Scripts\activate       # Windows
8
9 # Install into the active venv (pip-compatible interface)
10 uv pip install requests pandas
11 uv pip install -r requirements.txt
12
13 # Freeze current environment
14 uv pip freeze > requirements.txt
```

7 Running One-off Tools

```
1 # Run a tool without installing it permanently
2 uvx ruff check .
3 uvx black .
4
5 # Install a tool globally
6 uv tool install ruff
```

8 The 5 Commands You Will Use 90% of the Time

1. `uv init` — start a project
2. `uv add <pkg>` — add a dependency
3. `uv run <cmd>` — run code in the environment
4. `uv sync` — reproduce the environment from the lockfile
5. `uv pip install` — pip-style installs when you need them

9 Practice Task

Create a project, add `requests`, write a 3-line script that fetches a URL, and run it with `uv run`. Then delete `.venv` and run `uv sync` to confirm reproducibility.